TÀI LIỆU THIẾT KẾ DATABASE

Đồ án môn học: Cơ sở dữ liệu phân tán

Đề tài 4: Ứng dụng Gọi xe theo Vị trí

Nhóm Database (Nhóm 3)

1. Tổng quan hệ thống

Ứng dụng gọi xe hoạt động theo mô hình CSDL phân tán địa lý nhằm giảm độ trễ mạng cho người dùng tại từng khu vực. Dữ liệu được phân mảnh theo thành phố (HCM / HN), mỗi khu vực có 1 server chính (Primary) và 1 server dự phòng (Replica).

1.1. Kiến trúc tổng thể

Server	Cổng	Vai trò	Phục vụ
serverMienNam	:5001	Primary HCM	Read + Write
backupMienNam	:6001	Replica HCM	Read-only (backup)
serverMienBac	:5002	Primary HN	Read + Write
backupMienBac	:6002	Replica HN	Read-only (backup)

1.2. Nguyên tắc phân mảnh dữ liệu (Sharding)

- Người dùng ở HCM → ghi/đọc vào Primary HCM (:5001)
- Người dùng ở HN → ghi/đọc vào Primary HN (:5002)
- Cột thanh_pho trong bảng users và trips là khóa phân mảnh
- Mỗi Primary tự động đồng bộ sang Replica tương ứng (Master-Slave Replication)

2. Thiết kế Schema — Các bảng cốt lõi (4 bảng bắt buộc)

Đây là 4 bảng tối thiểu cần thiết để hệ thống hoạt động được. Nhóm Backend có thể bắt đầu viết API dựa trên 4 bảng này.

2.1. Bảng users (Người dùng)

Tên cột	Kiểu dữ liệu	Mô tả
id	UUID (PK)	Khóa chính, tự sinh
ho_ten	VARCHAR(100)	Họ và tên người dùng

Tên cột	Kiểu dữ liệu	Mô tả
email	VARCHAR(150) UNIQUE	Email đăng nhập, không trùng
mat_khau	VARCHAR(255)	Mật khẩu đã mã hóa (bcrypt)
so_dien_thoai	VARCHAR(15)	Số điện thoại
thanh_pho	ENUM('HCM','HN')	Thành phố — khóa phân mảnh
ngay_tao	TIMESTAMP	Ngày tạo tài khoản

2.2. Bảng trips (Chuyến đi)

Tên cột	Kiểu dữ liệu	Mô tả
id	UUID (PK)	Khóa chính
ma_nguoi_dung	UUID (FK → users)	Người đặt chuyến
vi_do_diem_don	DECIMAL(10,8)	Vĩ độ điểm đón (GPS)
kinh_do_diem_don	DECIMAL(11,8)	Kinh độ điểm đón (GPS)
dia_chi_diem_don	VARCHAR(255)	Địa chỉ điểm đón (text)
vi_do_diem_den	DECIMAL(10,8)	Vĩ độ điểm đến (GPS)
kinh_do_diem_den	DECIMAL(11,8)	Kinh độ điểm đến (GPS)
dia_chi_diem_den	VARCHAR(255)	Địa chỉ điểm đến (text)
trang_thai	ENUM	cho_xu_ly / hoan_thanh / da_huy
thanh_pho	ENUM('HCM','HN')	Thành phố — khóa phân mảnh
ngay_tao	TIMESTAMP	Ngày tạo chuyến

2.3. Bảng drivers (Tài xế)

Tên cột	Kiểu dữ liệu	Mô tả
id	UUID (PK)	Khóa chính
ho_ten	VARCHAR(100)	Họ và tên tài xế
so_dien_thoai	VARCHAR(15)	Số điện thoại
vi_do_hien_tai	DECIMAL(10,8)	Vĩ độ vị trí hiện tại
kinh_do_hien_tai	DECIMAL(11,8)	Kinh độ vị trí hiện tại
thanh_pho	ENUM('HCM','HN')	Thành phố tài xế hoạt động
dang_ranh	BOOLEAN	TRUE = đang rảnh, FALSE = đang chạy
ngay_tao	TIMESTAMP	Ngày tạo tài khoản

2.4. Bảng trip_assignments (Phân công chuyến đi)

Tên cột	Kiểu dữ liệu	Mô tả
id	UUID (PK)	Khóa chính
ma_chuyen_di	UUID (FK → trips)	Chuyến đi được phân công
ma_tai_xe	UUID (FK → drivers)	Tài xế nhận chuyến
thoi_diem_phan_cong	TIMESTAMP	Khi tài xế được giao chuyến
thoi_diem_hoan_thanh	TIMESTAMP	Khi chuyến kết thúc (NULL nếu chưa xong)

3. Phân tích thực thể mở rộng — Đề xuất bổ sung

3.1. Bảng vehicles (Phương tiện)

Lý do cần thiết: Một tài xế có thể sở hữu nhiều xe. Loại xe ảnh hưởng trực tiếp đến loại chuyến đi và giá cước. Nếu không có bảng này, không thể phân biệt được xe máy với ô tô 4 chỗ hay 7 chỗ.

Tên cột	Kiểu dữ liệu	Mô tả
id	UUID (PK)	Khóa chính
ma_tai_xe	UUID (FK → drivers)	Tài xế sở hữu phương tiện này
loai_xe	ENUM	xe_may / o_to_4_cho / o_to_7_cho
bien_so	VARCHAR(20) UNIQUE	Biển số xe, không trùng
hang_xe	VARCHAR(50)	Honda, Toyota, Yamaha...
mau_xe	VARCHAR(30)	Màu sắc phương tiện
nam_san_xuat	SMALLINT	Năm sản xuất
dang_hoat_dong	BOOLEAN	Xe có đang được sử dụng không

- Quan hệ: drivers (1) → (nhiều) vehicles
- Khi đặt chuyến, hệ thống cần biết loại xe để khớp với yêu cầu của khách

3.2. Bảng ride_types (Loại dịch vụ xe)

Lý do cần thiết: Ứng dụng như Grab có GrabBike, GrabCar 4 chỗ, GrabCar 7 chỗ với giá cước khác nhau. Bảng này định nghĩa các loại dịch vụ và công thức tính giá.

Tên cột	Kiểu dữ liệu	Mô tả
id	UUID (PK)	Khóa chính
ten_loai	VARCHAR(50)	Xe máy / Ô tô 4 chỗ / Ô tô 7 chỗ
gia_co_ban	DECIMAL(10,2)	Giá mở cửa (VNĐ)
gia_moi_km	DECIMAL(10,2)	Giá mỗi km (VNĐ)
mo_ta	TEXT	Mô tả dịch vụ
dang_hoat_dong	BOOLEAN	Dịch vụ này còn cung cấp không

- Quan hệ: ride_types (1) → (nhiều) trips — mỗi chuyến đi thuộc 1 loại dịch vụ
- Cần thêm cột ma_loai_dich_vu (FK → ride_types) vào bảng trips

3.3. Bảng payments (Thanh toán)

Lý do cần thiết: Mỗi chuyến đi cần ghi nhận việc thanh toán — bằng tiền mặt hay ví điện tử, đã thanh toán chưa, số tiền thực tế bao nhiêu. Đây là thực thể bắt buộc trong bất kỳ hệ thống thương mại nào.

Tên cột	Kiểu dữ liệu	Mô tả
id	UUID (PK)	Khóa chính
ma_chuyen_di	UUID (FK → trips)	Chuyến đi tương ứng
so_tien	DECIMAL(12,2)	Số tiền thanh toán (VNĐ)
phuong_thuc	ENUM	tien_mat / vi_dien_tu / the_ngan_hang
trang_thai	ENUM	cho_thanh_toan / da_thanh_toan / hoan_tien
thoi_diem_thanh_toan	TIMESTAMP	Khi nào hoàn tất thanh toán
ma_giao_dich	VARCHAR(100)	Mã giao dịch từ cổng thanh toán

- Quan hệ: trips (1) → (1) payments — mỗi chuyến chỉ có 1 giao dịch thanh toán

3.4. Bảng ratings (Đánh giá)

Lý do cần thiết: Sau mỗi chuyến, người dùng đánh giá tài xế và ngược lại. Điểm đánh giá ảnh hưởng đến ưu tiên hiển thị tài xế. Không có bảng này, hệ thống không có cơ chế kiểm soát chất lượng.

Tên cột	Kiểu dữ liệu	Mô tả
id	UUID (PK)	Khóa chính
ma_chuyen_di	UUID (FK → trips)	Chuyến đi được đánh giá
nguoi_danh_gia	UUID (FK → users hoặc drivers)	Ai đánh giá
loai_nguoi_danh_gia	ENUM	nguoi_dung / tai_xe
diem_so	SMALLINT	Từ 1 đến 5 sao
nhan_xet	TEXT	Nhận xét bằng văn bản (tùy chọn)
ngay_danh_gia	TIMESTAMP	Thời điểm đánh giá

- Quan hệ: trips (1) → (nhiều) ratings — 1 chuyến có thể có 2 đánh giá (user đánh tài xế + tài xế đánh user)

3.5. Bảng driver_location_logs (Lịch sử vị trí tài xế)

Lý do cần thiết: Để chứng minh tài xế thực sự đã di chuyển theo lộ trình, phục vụ giải quyết khiếu nại và tính phí chính xác. Đây cũng là minh chứng cho tính năng realtime tracking.

Tên cột	Kiểu dữ liệu	Mô tả
id	BIGSERIAL (PK)	Khóa chính tự tăng (dùng int vì insert nhiều)
ma_tai_xe	UUID (FK → drivers)	Tài xế di chuyển
ma_chuyen_di	UUID (FK → trips, nullable)	Chuyến đi đang thực hiện (nếu có)
vi_do	DECIMAL(10,8)	Vĩ độ tại thời điểm ghi
kinh_do	DECIMAL(11,8)	Kinh độ tại thời điểm ghi
thoi_diem	TIMESTAMP	Thời điểm ghi nhận vị trí

- Bảng này sẽ có lượng dữ liệu rất lớn (ghi mỗi 5-10 giây trong lúc chạy)
- Trong demo: có thể seed sẵn vài bản ghi để minh họa, không cần realtime thật

3.6. Bảng notifications (Thông báo)

Lý do cần thiết: Hệ thống cần thông báo khi tài xế đã đến nơi đón, khi chuyến hoàn thành, khi bị hủy. Bảng này lưu lịch sử thông báo đã gửi, phục vụ debug và kiểm tra.

Tên cột	Kiểu dữ liệu	Mô tả
id	UUID (PK)	Khóa chính
nguoi_nhan	UUID (FK)	User hoặc Driver nhận thông báo
loai_nguoi_nhan	ENUM	nguoi_dung / tai_xe
tieu_de	VARCHAR(100)	Tiêu đề thông báo
noi_dung	TEXT	Nội dung chi tiết
loai_thong_bao	ENUM	dat_xe / huy_xe / hoan_thanh / he_thong
da_doc	BOOLEAN	Người dùng đã đọc chưa
ngay_gui	TIMESTAMP	Thời điểm gửi thông báo

4. Tổng hợp quan hệ giữa các thực thể (cho người vẽ ERD)

Bảng dưới đây liệt kê toàn bộ quan hệ cần thể hiện trong sơ đồ ERD. Tổng cộng có 10 thực thể và 12 quan hệ.

Thực thể A	Quan hệ	Thực thể B	Diễn giải
users	1 — nhiều	trips	1 user đặt nhiều chuyến
users	1 — nhiều	ratings	1 user để lại nhiều đánh giá
users	1 — nhiều	notifications	1 user nhận nhiều thông báo
drivers	1 — nhiều	vehicles	1 tài xế có nhiều xe
drivers	1 — nhiều	trip_assignments	1 tài xế nhận nhiều chuyến

Thực thể A	Quan hệ	Thực thể B	Diễn giải
drivers	1 — nhiều	driver_location_logs	1 tài xế có nhiều bản ghi vị trí
drivers	1 — nhiều	notifications	1 tài xế nhận nhiều thông báo
trips	1 — 1	payments	1 chuyến có đúng 1 thanh toán
trips	1 — nhiều	ratings	1 chuyến có tối đa 2 đánh giá
trips	1 — nhiều	trip_assignments	1 chuyến có thể được thử phân công lại
trips	1 — nhiều	driver_location_logs	1 chuyến ghi nhiều vị trí tài xế
ride_types	1 — nhiều	trips	1 loại xe được đặt trong nhiều chuyến

5. Phân tích Use Case đầy đủ

Hệ thống gọi xe có 3 tác nhân chính: Người dùng (hành khách), Tài xế, và Hệ thống (tự động). Tổng cộng có 20 use case được phân tích dưới đây.

5.1. Use Case phía Người dùng (8 use case)

Mã UC	Tên use case	Mô tả ngắn	Bảng liên quan
UC-01	Đăng ký tài khoản	Nhập họ tên, email, mật khẩu, chọn thành phố	users
UC-02	Đăng nhập	Xác thực email + mật khẩu	users
UC-03	Đặt chuyến xe	Chọn điểm đón, điểm đến, loại xe — hệ thống tìm tài xế	trips, ride_types
UC-04	Hủy chuyến	Hủy chuyến khi đang ở trạng thái chờ xử lý	trips
UC-05	Xem lịch sử chuyến	Xem danh sách các chuyến đã đi (hoàn thành hoặc đã hủy)	trips, payments
UC-06	Đánh giá tài xế	Cho điểm 1-5 sao sau khi chuyến hoàn thành	ratings
UC-07	Xem thông báo	Xem thông báo về tình trạng chuyến đi	notifications
UC-08	Cập nhật hồ sơ	Thay đổi số điện thoại, mật khẩu	users

5.2. Use Case phía Tài xế (7 use case)

Mã UC	Tên use case	Mô tả ngắn	Bảng liên quan
UC-09	Đăng ký tài xế	Tạo tài khoản tài xế, thêm thông tin phương tiện	drivers, vehicles
UC-10	Bật/tắt trạng thái rảnh	Tài xế bật sẵn sàng nhận chuyến hoặc nghỉ	drivers
UC-11	Nhận chuyến đi	Chấp nhận yêu cầu chuyến được hệ thống gửi đến	trip_assignments
UC-12	Từ chối chuyến	Không nhận chuyến này, hệ thống tìm tài xế khác	trip_assignments
UC-13	Cập nhật vị trí	Gửi tọa độ GPS liên tục khi đang chạy	driver_location_logs
UC-14	Hoàn thành chuyến	Đánh dấu chuyến đã xong, kích hoạt thanh toán	trips, trip_assignments, payments
UC-15	Đánh giá hành khách	Cho điểm hành khách sau chuyến	ratings

5.3. Use Case phía Hệ thống — tự động (5 use case)

Mã UC	Tên use case	Mô tả ngắn	Bảng liên quan
UC-16	Định tuyến theo vị trí	Tự động kết nối app vào server HCM hoặc HN dựa trên thành phố	Tầng kết nối DB
UC-17	Tìm tài xế gần nhất	Sau khi khách đặt xe, tìm tài xế rảnh gần nhất theo GPS	drivers, trip_assignments
UC-18	Tính giá chuyến	Dựa trên khoảng cách và loại xe để ra giá cước	trips, ride_types, payments
UC-19	Failover tự động	Khi Primary sập, chuyển kết nối sang Replica — chỉ đọc	Tầng kết nối DB
UC-20	Nhân bản dữ liệu	Primary tự đồng bộ dữ liệu sang Replica liên tục	Tất cả các bảng

Lưu ý: UC-16, UC-19, UC-20 là 3 use case đặc trưng của CSDL PHÂN TÁN — đây là điểm khác biệt so với hệ thống thông thường, cần nhấn mạnh trong báo cáo.

6. SQL Tạo Bảng — Đầy đủ

Bao gồm 4 bảng cốt lõi + 3 bảng mở rộng quan trọng nhất (vehicles, payments, ratings). Chạy trên cả 4 server.

```
-- ==============================================
-- BẢNG CỐT LÕI
-- ==============================================

-- Bảng người dùng
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ho_ten VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    mat_khau VARCHAR(255) NOT NULL,
    so_dien_thoai VARCHAR(15),
    thanh_pho VARCHAR(10) CHECK (thanh_pho IN ('HCM','HN')),
    ngay_tao TIMESTAMP DEFAULT NOW()
);
```

```
-- Bảng loại dịch vụ xe (thêm mới)
CREATE TABLE ride_types (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ten_loai VARCHAR(50) NOT NULL,
    gia_co_ban DECIMAL(10,2) NOT NULL,
    gia_moi_km DECIMAL(10,2) NOT NULL,
    mo_ta TEXT,
    dang_hoat_dong BOOLEAN DEFAULT TRUE
);
```

```
-- Bảng chuyến đi (cập nhật: thêm loại dịch vụ)
CREATE TABLE trips (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ma_nguoi_dung UUID REFERENCES users(id),
    ma_loai_dich_vu UUID REFERENCES ride_types(id),
    vi_do_diem_don DECIMAL(10,8),
    kinh_do_diem_don DECIMAL(11,8),
    dia_chi_diem_don VARCHAR(255),
    vi_do_diem_den DECIMAL(10,8),
```

```
    kinh_do_diem_den DECIMAL(11,8),
    dia_chi_diem_den VARCHAR(255),
    trang_thai VARCHAR(20) DEFAULT 'cho_xu_ly'
        CHECK (trang_thai IN ('cho_xu_ly','hoan_thanh','da_huy')),
    thanh_pho VARCHAR(10) CHECK (thanh_pho IN ('HCM','HN')),
    ngay_tao TIMESTAMP DEFAULT NOW()
);
```

```
-- Bảng tài xế
CREATE TABLE drivers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ho_ten VARCHAR(100) NOT NULL,
    so_dien_thoai VARCHAR(15),
    vi_do_hien_tai DECIMAL(10,8),
    kinh_do_hien_tai DECIMAL(11,8),
    thanh_pho VARCHAR(10) CHECK (thanh_pho IN ('HCM','HN')),
    dang_ranh BOOLEAN DEFAULT TRUE,
    ngay_tao TIMESTAMP DEFAULT NOW()
);
```

```
-- Bảng phân công chuyến đi
CREATE TABLE trip_assignments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ma_chuyen_di UUID REFERENCES trips(id),
    ma_tai_xe UUID REFERENCES drivers(id),
    thoi_diem_phan_cong TIMESTAMP DEFAULT NOW(),
    thoi_diem_hoan_thanh TIMESTAMP
);
```

```
-- ==============================================
-- BẢNG MỞ RỘNG
-- ==============================================

-- Bảng phương tiện
CREATE TABLE vehicles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ma_tai_xe UUID REFERENCES drivers(id),
    loai_xe VARCHAR(20) CHECK (loai_xe IN ('xe_may','o_to_4_cho','o_to_7_cho')),
    bien_so VARCHAR(20) UNIQUE NOT NULL,
    hang_xe VARCHAR(50),
    mau_xe VARCHAR(30),
    nam_san_xuat SMALLINT,
    dang_hoat_dong BOOLEAN DEFAULT TRUE
);

-- Bảng thanh toán
CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ma_chuyen_di UUID UNIQUE REFERENCES trips(id),
    so_tien DECIMAL(12,2) NOT NULL,
    phuong_thuc VARCHAR(20) CHECK (phuong_thuc IN
('tien_mat','vi_dien_tu','the_ngan_hang')),
    trang_thai VARCHAR(20) DEFAULT 'cho_thanh_toan'
        CHECK (trang_thai IN ('cho_thanh_toan','da_thanh_toan','hoan_tien')),
    thoi_diem_thanh_toan TIMESTAMP,
    ma_giao_dich VARCHAR(100)
);

-- Bảng đánh giá
CREATE TABLE ratings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ma_chuyen_di UUID REFERENCES trips(id),
    nguoi_danh_gia UUID NOT NULL,
    loai_nguoi_danh_gia VARCHAR(10) CHECK (loai_nguoi_danh_gia IN
('nguoi_dung','tai_xe')),
```

```
    diem_so SMALLINT CHECK (diem_so BETWEEN 1 AND 5),
    nhan_xet TEXT,
    ngay_danh_gia TIMESTAMP DEFAULT NOW()
);
```

7. Cơ chế Phân tán & Nhân bản

7.1. Nhân bản dữ liệu (Replication)

- Loại: Master-Slave Replication (Primary ghi, Replica đọc)
- Cấu hình bằng tính năng có sẵn của PostgreSQL (streaming replication) hoặc MySQL (binary log replication)
- Replica tự động nhận dữ liệu từ Primary theo thời gian thực
- Replica chỉ cho phép SELECT, không cho INSERT / UPDATE / DELETE

7.2. Cơ chế Failover

- Khi Primary HCM sập → ứng dụng tự kết nối sang Replica HCM (:6001)
- Khi Primary HN sập → ứng dụng tự kết nối sang Replica HN (:6002)
- Logic failover đã có sẵn trong api_service.dart của nhóm FE
- Sau khi chuyển sang Replica: chỉ đọc được lịch sử chuyến, không đặt chuyến mới

7.3. Bảng so sánh trạng thái bình thường vs. khi Primary sập

Thao tác	Bình thường (Primary)	Khi Primary sập (Replica)
Xem lịch sử chuyến	Được	Vẫn được
Đăng nhập / xem hồ sơ	Được	Vẫn được
Đặt chuyến mới	Được (ghi Primary)	KHÔNG được — báo lỗi rõ
Cập nhật trạng thái	Được	KHÔNG được
Xem thông báo cũ	Được	Vẫn được

8. Checklist Công Việc Nhóm Database

Giai đoạn 1 — Thiết kế

1. Xác nhận schema 10 bảng với nhóm BE (ưu tiên 4 bảng cốt lõi trước)
2. Vẽ sơ đồ ERD đầy đủ với 10 thực thể và 12 quan hệ (dựa vào Phần 4)
3. Vẽ sơ đồ Use Case với 20 use case (dựa vào Phần 5)
4. Hoàn thiện file schema.sql

Giai đoạn 2 — Cài đặt

5. Cài Docker, viết docker-compose.yml dựng 4 container DB
6. Cấu hình replication Primary → Replica cho cả 2 cụm

7. Test kết nối được từ backend vào đủ 4 cổng

Giai đoạn 3 — Kiểm thử

8. Test bình thường: đặt chuyến → lưu vào Primary → hiển thị lịch sử
9. Test failover: tắt Primary HCM → app chuyển sang :6001 tự động
10. Test read-only: thử đặt chuyến khi dùng Replica → phải báo lỗi
11. Test dữ liệu nhất quán sau khi khôi phục Primary

Giai đoạn 4 — Bàn giao cho nhóm BE

12. Giao file docker-compose.yml + schema.sql
13. Giao tài liệu này kèm connection strings
14. Tạo seed data mẫu (5-10 users, trips, drivers mỗi cụm)

9. Thông tin bàn giao cho Nhóm Backend

Connection strings (PostgreSQL)

```
# Primary HCM — Read + Write
postgresql://admin:password@localhost:5001/rideapp_hcm

# Replica HCM — Read-only
postgresql://admin:password@localhost:6001/rideapp_hcm

# Primary HN — Read + Write
postgresql://admin:password@localhost:5002/rideapp_hn

# Replica HN — Read-only
postgresql://admin:password@localhost:6002/rideapp_hn
```

Quy tắc quan trọng cho Backend

- Luôn đọc thanh_pho của user trước khi chọn server kết nối
- Mọi thao tác ghi (INSERT, UPDATE) → dùng Primary tương ứng
- Khi Primary lỗi → bắt exception, thử lại với Replica (đọc được, không ghi)
- Trả về HTTP 503 kèm thông báo tiếng Việt khi không ghi được vào Replica